MAAgentManager 优化计划

当前状态

MAActorSubsystem 目前是一个简单的 **Actor 注册表/工厂**：

- 生成/销毁 Character 和 Sensor
- 查询接口 (GetByType, GetByID, GetByName)
- 简单批量操作 (StopAll, MoveAllTo)

优化目标：参考 CARLA TrafficManager

将 MAActorSubsystem 升级为 **集群管理器**，类似 CARLA 的 TrafficManager。

对比

功能	CARLA TrafficManager	当前 MAActorSubsystem	优化后
生命周期管理	✅	✅	✅
查询接口	✅	✅	✅
编队管理	-	❌	✅
集群协调	✅ 路径/避碰	❌	✅
行为控制	✅ 下发指令	❌	✅
全局参数	✅ 速度/距离	❌	✅

新增功能

1. 编队管理 (Formation Management)

```
// 启动编队 - Leader 带领所有机器人
void StartFormation(AMACharacter* Leader, EFormationType Type);

// 停止编队
```

```
void StopFormation();

// 机器人加入/离开编队
void JoinFormation(AMACharacter* Robot);
void LeaveFormation(AMACharacter* Robot);

// 切换编队类型
void SetFormationType(EFormationType Type);
```

2. 集群协调 (Swarm Coordination)

```
// 定时更新 - 计算每个机器人的目标位置
void Tick(float DeltaTime);

// 计算编队位置（世界坐标）
TArray<FVector> CalculateFormationPositions();

// 下发导航指令给机器人
void DispatchNavigateCommands();
```

3. 全局参数 (Global Parameters)

```
// 编队间距
UPROPERTY(EditAnywhere)
float FormationSpacing = 200.f;

// 更新频率
UPROPERTY(EditAnywhere)
float UpdateInterval = 0.3f;

// 移动速度倍率
UPROPERTY(EditAnywhere)
float GlobalSpeedMultiplier = 1.0f;
```

架构设计

```
┌──────────────────────────────────────────────────────────┐
│                MAActorSubsystem                          │
│  ┌──────────────────┐  ┌──────────────────────────┐     │
│  │  Actor Factory   │  │  Formation Manager       │     │
│  │  - Spawn         │  │  - Leader                │     │
│  │  - Destroy       │  │  - Members[]             │     │
│  │  - Query         │  │  - Type                  │     │
│  └──────────────────┘  │  - CalculatePositions()  │     │
│                        │  - DispatchCommands()    │     │
```

```
│                        └──────────────────────┘   │
└──────────────────────────────────────────────────┘
                          │
                          │ TryNavigateTo(Position)
                          ▼
        ┌─────────────────┼─────────────────┐
        │                 │                 │
   ┌────▼────┐       ┌────▼────┐       ┌────▼────┐
   │ Robot_0 │       │ Robot_1 │       │ Robot_2 │
   │Navigate │       │Navigate │       │Navigate │
   └─────────┘       └─────────┘       └─────────┘
```

机器人端简化

优化后，机器人只需要 **Navigate 能力**（基于 NavMesh 的导航）：

```
// 机器人不再需要理解编队逻辑
// 只需响应 Subsystem 下发的导航指令
// Subsystem 调用机器人的 TryNavigateTo，内部使用 AIController->MoveToLocation
void AMARobotDogCharacter::TryNavigateTo(FVector TargetLocation)
{
    // 使用 NavMesh 导航到目标位置
    AAIController* AICtrl = Cast<AAIController>(GetController());
    if (AICtrl)
    {
        AICtrl->MoveToLocation(TargetLocation, AcceptanceRadius);
    }
}
```

Navigate vs MoveTo 的区别：

- Navigate：使用 NavMesh 寻路，绕过障碍物
- MoveTo：直线移动，可能卡住

实现步骤

1. [x] 在 MAActorSubsystem 中添加编队状态变量
2. [x] 实现 StartFormation / StopFormation
3. [x] 实现 CalculateFormationPositions (从 GA_Formation 迁移)
4. [x] 添加定时器更新逻辑 (UpdateFormation)
5. [x] 实现下发导航指令 (TryNavigateTo)
6. [ ] 简化 GA_Formation 或移除（改用 Subsystem 直接控制）
7. [x] 更新 PlayerController 调用方式

按键说明

按 B 键循环切换：

- 第1次：Line 编队
- 第2次：Column 编队
- 第3次：Wedge 编队
- 第4次：Diamond 编队
- 第5次：Circle 编队
- 第6次：停止编队
- 第7次：Line 编队（循环）

滞后区机制 (Hysteresis)

为避免机器人在编队位置附近反复启停（抖动），使用滞后区：

```
距离
  ▲
  │
150├──────────────────── StartMoveThreshold (开始移动)
  │     ↓ 开始移动
  │
 80├──────────────────── StopMoveThreshold (停止移动)
  │     ↓ 停止移动
  0└──────────────────────────────────────────►
```

- `FormationStartMoveThreshold = 150.f` - 距离超过此值才开始移动
- `FormationStopMoveThreshold = 80.f` - 距离小于此值才停止移动
- 中间区域保持当前状态，避免抖动

优势

- **职责分离**：机器人只负责移动，Subsystem 负责集群逻辑
- **易于扩展**：添加新编队类型只需修改 Subsystem
- **动态调整**：机器人加入/离开时自动重新计算位置
- **统一管理**：全局参数集中配置